

DATA SCIENCE PROJECT

ROADSENSE

다중 교통사고 데이터를 활용한
치명 사고 분류 및 위험 요인 분석

Individual Project

Data Analysis · Feature Engineering · Machine Learning

Python · Pandas · scikit-learn · XGBoost

01. Project Overview

4개의 테이블을 사고 단위 192개 피처로 재구성하고
OOF 기반 검증을 통해 치명 사고 탐지 모델을 설계하였습니다.

문제 정의

교통사고의 심각도는 장소, 도로 환경, 인적 특성, 차량 및 충돌 조건이 복합적으로 작용한 결과로 나타납니다. 치명률이 높은 사고는 직관적으로 파악하기 어렵고 사고 발생 이전 예측할 수 있는 근거도 명확하지 않습니다.

본 프로젝트는 사고·장소·인적·차량 정보를 사고 단위로 통합한 데이터를 바탕으로 치명 사고와 연관된 주요 조건을 식별하고 사고 심각도를 예측할 수 있는지 검증하는 것을 목표로 합니다.

프로젝트 목표

- 서로 다른 데이터 표현과 범주를 일관된 기준으로 정리
- Users와 Vehicles 데이터셋을 사고 단위 피처로 집계
- 네 개 데이터셋을 병합하고 데이터 정합성 검증
- 불균형 분류에 적합한 모델·평가지표·임계값 선정

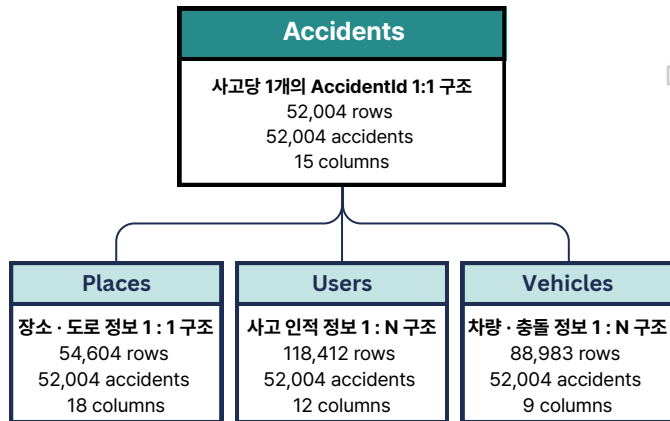
항 목

내 용

수행 형태	개인 프로젝트
문제 유형	불균형 이진 분류
최종 데이터	47,821건, 모델 피처 192개
검증	Train: Stratified 5-Fold CV · Hold-out Test
비교 모델	RandomForest · XGBoost
최종 성능	Recall 0.4192 · F1 0.3011 · ROC-AUC 0.8224 · AP 0.2326
추가 실험	SMOTENC 오버샘플링 시도

02. Multi-table Data Structure

52,004건의 사고를 기준으로 장소·사용자·차량 정보가
1:1 및 1:N 구조로 연결된 관계형 데이터입니다.



[2]: # 1. 데이터 로드 및 변수 저장

```
accidents_train = pd.read_csv('./data/accidents_train.csv')
places_train = pd.read_csv('./data/places_train.csv')
users_train = pd.read_csv('./data/users_train.csv')
vehicles_train = pd.read_csv('./data/vehicles_train.csv')

accidents_test = pd.read_csv('./data/accidents_test.csv')
places_test = pd.read_csv('./data/places_test.csv')
users_test = pd.read_csv('./data/users_test.csv')
vehicles_test = pd.read_csv('./data/vehicles_test.csv')
```

01_RoadSense_DataPreprocessing.ipynb

데이터셋 요약 (Train 기준)

Dataset	Rows	Unique AccidentId	Relationship
Accidents	52,004	52,004	1:1 (기준테이블)
Places	54,604	52,004	1:1 (사고 발생 장소)
Users	118,412	52,004	1:N (사고당 관련 인물)
Vehicles	88,983	52,004	1:N (사고당 여러 차량)

Analysis Decision

최종적으로 하나의 데이터셋을 완성하기 위해
Users와 Vehicles를 AccidentId 기준으로
집계하여 사고당 한 행으로 만든 뒤
Accidents·Places 데이터셋과 병합하여
네 개의 테이블을 하나로 통합했습니다.

Why It Matters

AccidentId 기준으로 집계하지 않고
Users와 Vehicles를 그대로 병합하면
사고 단위가 유지되지 않고 과대 반영되어
위험도 분석과 탐지 기준이 왜곡될 수 있습니다.
분석 단위를 사고로 통일하여 각 사고에 동일한
가중치를 부여하고 모델 결과를 개별 사고 단위로
해석할 수 있도록 설계했습니다.

03. Data Quality & Preprocessing

Preprocessing 단계에서는 표준화되지 않은 컬럼의 라벨들을 정제하여 데이터 품질을 확보하는 작업을 진행합니다.

1. Date Column 표준화

```
[20]: # Date 데이터 조회
accidents_train["Date"].head(10)
```

```
[20]: 0          24/01/2018
      1          Feb 12 2018
      2          04-Mar-18
      3          May 05 2018
      5    2018-11-30 00:00:00
      6          Feb 18 2018
      8          28/05/2018
      9          May 31 2018
     10          15/06/2018
     11    2018-07-19 00:00:00
      Name: Date, dtype: str
```

상위 10개의 데이터를 조회한 결과 날짜 표기에 사용된 구분자와 형식이 일관되지 않음을 확인했습니다. 또한 '년'도는 모두 2018년으로 확인되며 이를 표준화 하기 위해 날짜 형식을 YYYY-MM-DD로 통일하겠습니다.

```
[21]: # 약어 문자 숫자와 매핑
month_map = {
    'Jan': '01', 'Feb': '02', 'Mar': '03', 'Apr': '04',
    'May': '05', 'Jun': '06', 'Jul': '07', 'Aug': '08',
    'Sep': '09', 'Oct': '10', 'Nov': '11', 'Dec': '12'}
```

```
[22]: # 약어 문자 숫자로 변환
accidents_train['Date'] = accidents_train['Date'].replace(month_map, regex=True)
```

먼저 '월'의 표기가 약어로 되어있는 경우를 숫자로 변환을 진행합니다.
Date 컬럼의 경우 시간이 표기되어있지 않은 날짜는 날짜 포맷의 맨 뒷부분이 연도로 확인됩니다.
우선 시간이 표기되어있지 않은 행을 먼저 찾아 YYYY-MM-DD 형식을 맞춰주겠습니다.

```
# 시간 미포함 mask 저장
mask_no_time = ~accidents_train["Date"].str.contains(r"\d{2}:\d{2}:\d{2}", na=False)
```

```
# 구분자 통일
accidents_train.loc[mask_no_time, "Date"] = (accidents_train.loc[mask_no_time, "Date"].astype(str).str.replace(r"[\/ ]", "-", regex=True))
```

```
# 18년도로 되어있는 년도를 2018로 수정
```

```
mask_18 = mask_no_time & accidents_train["Date"].str.contains(r"(?!20)18$", na=False)
accidents_train.loc[mask_18, "Date"] = (accidents_train.loc[mask_18, "Date"].str.replace(r"18$", "2018", regex = True))
```

```
# 2018년도를 맨앞으로 이동하기
```

```
accidents_train.loc[mask_no_time, "Date"] = (
    accidents_train.loc[mask_no_time, "Date"].str.replace(r"^(?!\d{2})-(\d{2})-(2018)$",
        r"\3-\1-\2",
        regex=True))
```

```
# 월'이 12초과인 경우 mask 저장
mask_invalid_month = (
    mask_no_time &
    (accidents_train["Date"].str.extract(r"^(2018-(\d{2})-(\d{2}))")[0].astype(int) > 12))

# '월' 12 초과 자리 교체
accidents_train.loc[mask_invalid_month, "Date"] = (
    accidents_train.loc[mask_invalid_month, "Date"].str.replace(r"^(2018-(\d{2})-(\d{2}))$",
                                                                r"2018-\2-\1",
                                                                regex=True))

# 00:00:00 포맷 삭제하기
accidents_train["Date"] = (accidents_train["Date"].astype(str).str.replace(r"\s+\d{2}:\d{2}:\d{2}$",
                                                                "",
                                                                regex=True))
```

먼저 시간 정보가 없는 행을 'mask_no_time' 객체에 저장하고 정규표현식을 이용해 구분자를 변환합니다.
 날짜 맨 뒷부분의 연도가 YY 형태인 경우를 찾아 YYYY 형태로 변환하고 연도를 맨 앞으로 이동합니다.
 날짜 중간 부분은 '월'에 해당하므로 값이 12를 초과할 경우 맨 뒷자리와 교체합니다.
 시간 정보가 포함된 날짜는 이미 YYYY-MM-DD 형식이므로 뒷부분의 시간 값만 삭제 처리합니다.

```
[31]: # 최종 변환 확인
accidents_train["Date"].head(10)

[31]: 0      2018-01-24
      1      2018-02-12
      2      2018-04-03
      3      2018-05-05
      5      2018-11-30
      6      2018-02-18
      8      2018-05-28
      9      2018-05-31
     10      2018-06-15
     11      2018-07-19
      Name: Date, dtype: str
```

2. 형식·언어 불일치 진단 및 표준화

```
[100]: # 고유 값 확인
users_train['SafetyDeviceUsed'].unique()

[100]: <StringArray>
['Yes', '1', 'True', nan, 'Oui', 'Y', 'Unknown', 'N', 'Non', '0', 'False',
 'No']
Length: 12, dtype: str

[101]: yes_values = ['Yes', '1', 'True', 'Oui', 'Y']
      no_values = ['No', 'N', 'Non', '0', 'False']

      users_train['SafetyDeviceUsed'] = users_train['SafetyDeviceUsed'].apply(
          lambda x: 'Yes' if x in yes_values else ('No' if x in no_values else x))

[102]: # 변경 고유 값 확인
users_train['SafetyDeviceUsed'].unique()

[102]: <StringArray>
['Yes', nan, 'Unknown', 'No']
Length: 4, dtype: str
```

01_RoadSense_DataPreprocessing.ipynb

고유값 확인시 다양한 언어와 표기법 또는 불용어로 일관성 없는 라벨로 되어 있었습니다.
 'Yes'와 'No'의 같은 의미를 가지고 있는 라벨들을 모아 lambda로 라벨을 통일하였으며
 기존의 'nan'과 'Unknown'은 다른 의미로 분류하기 위해 기존의 라벨을 유지하였습니다.

3. 오염값 'Modified' 처리하기

[62]: #AccidentId 중복행 확인해보기 (AccidentId를 기준으로 오름차순 정렬)

places_train[places_train['AccidentId'].duplicated(keep=False)].sort_values('AccidentId').head(20)

	AccidentId	RoadType	RoadNumber	RoadSecNumber	RoadLetter	Circulation	LaneNumber	SpecialLane	Slope	RoadMarkerId	RoadMarkerDistance	Layout	StripWidth	LaneWidth	Su
	16	201800000019	Departmental	145	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN	Straight	NaN	NaN
	53801	201800000019	Departmental	Modified	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN	Straight	NaN	NaN
	52350	201800000020	Modified	NaN	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN	Unknown	NaN	NaN
	17	201800000020	Communal	NaN	NaN	NaN	TwoWay	2.0	0	Flat	NaN	NaN	LeftCurve	NaN	NaN
	91	201800000100	Departmental	115	NaN	NaN	TwoWay	2.0	0	Flat	5.0	1000.0	Straight	NaN	NaN
	53107	201800000100	Departmental	115	NaN	NaN	TwoWay	2.0	0	Flat	5.0	1000.0	Modified	NaN	NaN

Places 데이터셋에서 .duplicated()을 사용하여 중복행을 제거하였으나 AccidentId가 중복되는 경우가 확인됩니다.
Places는 AccidentId 하나당 하나의 정보만 담고 있어야 하는 테이블입니다.
확인 결과 중복행 대부분은 컬럼 값이 동일했지만 일부 경우에는 'Modified'라는 문자열이 나타났습니다.
'Modified'는 정상 범주에 속하지 않는 값으로 데이터 처리 과정에서 발생한 오염값일 가능성이 있다고 판단했습니다.
또한 Modified가 포함되지 않은 중복행들은 다른 결측 패턴을 보이며 상호 보완하는 관계에 있었습니다.

```
[67]: # 'Modified'는 오염값으로 간주해 결측 처리
places_train = places_train.replace('Modified', np.nan)

# 정보 부재 표기를 결측값 세 컬럼에 적용
cols = ['Layout', 'SurfaceCondition', 'Infrastructure']

places_train[cols] = places_train[cols].replace(['-', '0', 'Unknown'], np.nan)

# 동일 AccidentId 안에서 서로 다른 비결측값이 존재하는 컬럼 확인
duplicate_places = places_train[places_train['AccidentId'].duplicated(keep=False)]
conflict_by_column = (
    duplicate_places.groupby('AccidentId')
    .nunique(dropna=True)
    .gt(1)
    .sum()
    .sort_values(ascending=False)
)
```

중복 AccidentId 수: 1,835
서로 다른 비결측값이 존재하는 사고 수(컬럼별):
Series([], dtype: int64)

```
[68]: # 사고별 첫 번째 비결측값을 선택해 1행으로 통합
places_train = places_train.groupby('AccidentId', as_index=False).first()

assert places_train['AccidentId'].is_unique, 'Places의 AccidentId가 유일하지 않습니다.'
print(f"통합 후 Places 행 수: {len(places_train):,}")
```

통합 후 Places 행 수: 52,004

```
[69]: # 병합 후 행 개수 확인 (병합 전 AccidentId 고유값과 같아야함)
print(len(places_train))
```

52004

이를 처리하기 위해 먼저 'Modified'는 오염된 값으로 판단하여 'NaN'으로 변경해 결측으로 처리했습니다.
중복행을 하나의 행으로 통합하기 전 동일한 AccidentId 내에서 서로 다른 비결측값이 존재하는지 조회합니다.
각 컬럼마다 서로 다른 비결측값이 없음을 확인했으며 컬럼별로 비결측값을 가진 위치가 달랐기 때문에
.first()를 사용해 중복행의 각 컬럼에서 비결측값을 선택하여 1행으로 통합했습니다.
통합 후 AccidentId의 고유값과 places 행의 개수가 동일한것을 확인하였습니다.

이외 나머지 컬럼들에 대해서도 각 특성값을 표준화하고
결측치와 오류값들을 찾아 데이터가 담고있는 정보를 찾아 데이터를 정의하였습니다.

04. EDA & Feature Engineering

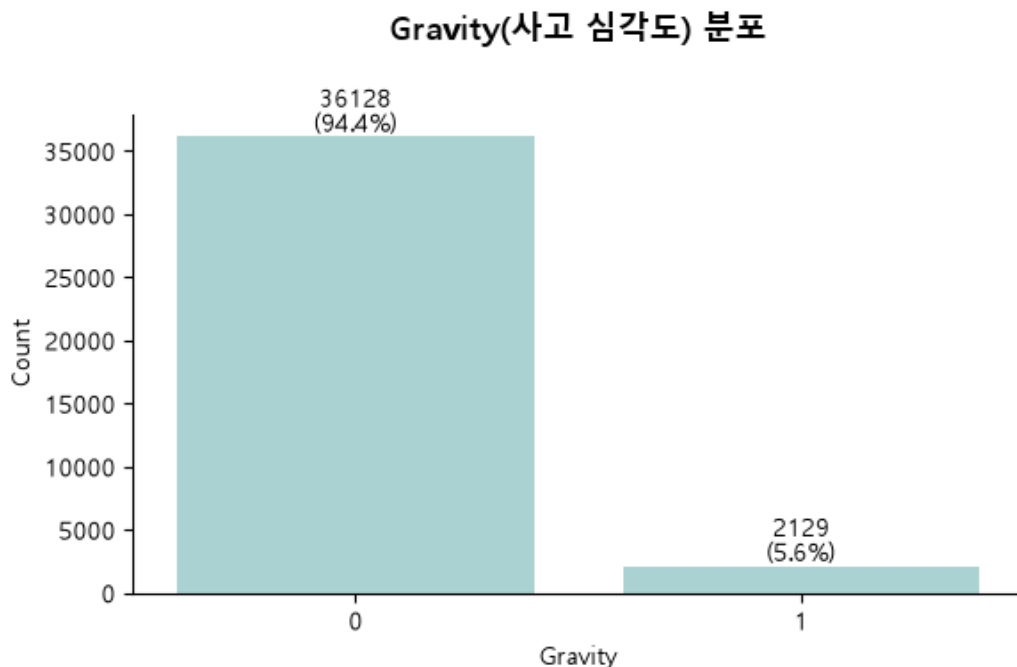
치명 사고 여부를 기준으로 각 컬럼의 라벨 분포와 비율을 확인하여
희소하거나 세분화된 라벨을 재정의의를 통해 의미 있는 범주로 통합하였습니다.

1. Train 데이터와 Test 데이터 분리

- Kaggle에서 제공한 Test 데이터에는 타깃값이 없어 기존 Train 데이터를 Train과 Test으로 분리하였습니다.
- Accidents 컬럼을 먼저 분리 후 AccidentId를 기준으로 Places, Users, Vehicles의 사고 기록을 나누었습니다.
- 동일한 사고에 속한 정보가 모두 같은 세트에 포함되어 이후 사고 단위로 집계·결합될 수 있도록 하였습니다.
- 데이터누수 방지를 위해 EDA 및 재분류 기준은 Train 데이터에서 수립하고 Test 데이터에도 동일하게 적용하였습니다.

2. Target Data : Gravity

이 프로젝트에서 Target 값인 Gravity는 NonLethal과 Lethal로 구분되어 있습니다.



NonLethal은 약 94.4%, Lethal은 5.56%로 심각하게 편향된 분포를 보입니다.

이는 모델 학습 시에도 Lethal 데이터가 희소해 Target 값을 제대로 구분해내지 못할 가능성을 시사합니다.

이런 불균형 상황에서는 모델이 다수 클래스로 편향되기 쉬우므로 Accuracy만으로 성능을 평가하기 어렵습니다.

따라서 Precision, Recall, F1-score 등 소수 클래스인 Lethal의 탐지 성능을 평가하는 지표를 함께 사용할 예정입니다.

2. 시간 관련 컬럼 EDA

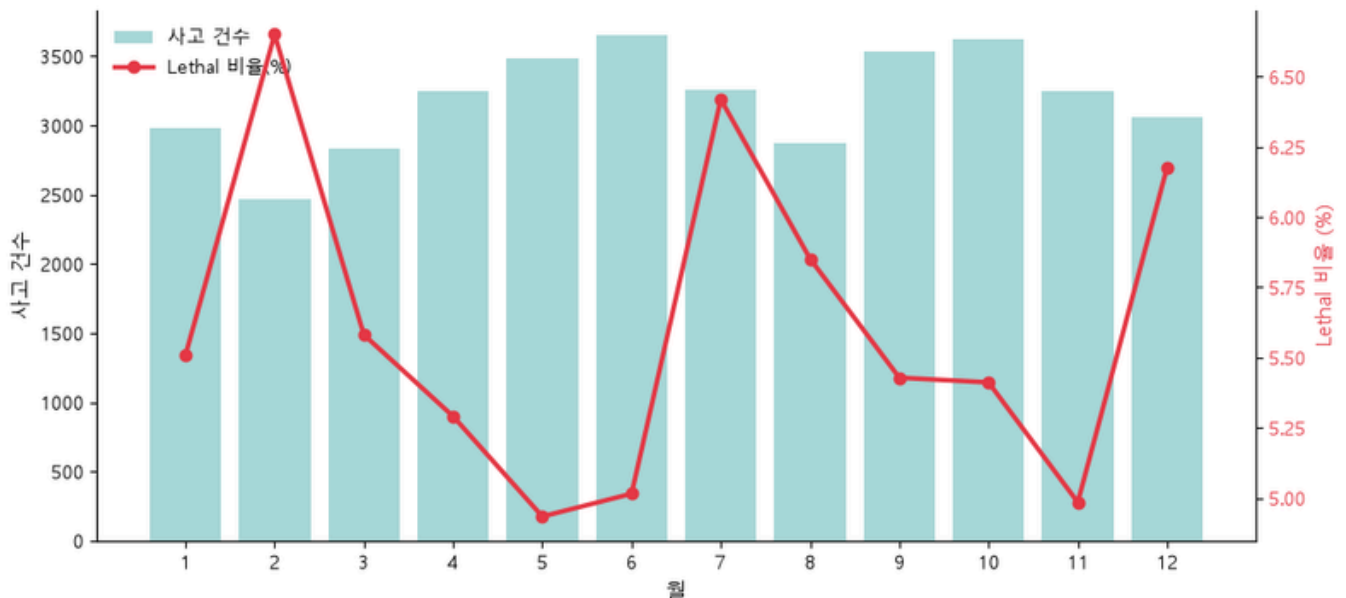
Date와 Hour 컬럼은 시간 관련 컬럼입니다.

원본 Date는 개별 사건의 식별자의 가깝고 그 자체로는 통계적으로 반복되는 패턴을 찾기 어렵습니다.
이에 월별·요일별·시간별로 분류하여 계절성, 요일 패턴, 시간대별 경향 등 사고 발생과 관련된
주기적 요인을 확인하고 타깃 데이터와의 관계를 반영한 새로운 피처를 생성하겠습니다.

월별, 요일별, 시간대별로 분리한 변수 생성

```
accidents_train['Date'] = pd.to_datetime(accidents_train['Date'])
accidents_train['Month'] = accidents_train['Date'].dt.month
accidents_train['DayOfWeek'] = accidents_train['Date'].dt.day_name()
accidents_train['HourOfDay'] = accidents_train['Hour'].str.split(':').str[0].astype(int)
```

월별 사고 건수 및 Lethal 비율



[13]: # 절대 건수(Lethal count) 월별 확인

```
monthly_lethal_count = accidents_train.groupby('Month')['Gravity'].apply(lambda x: (x == 1).sum())
print(monthly_lethal_count)
```

```
Month
1      164
2      164
3      158
4      172
5      172
6      183
7      209
8      168
9      192
10     196
11     162
12     189
Name: Gravity, dtype: int64
```

월별 사고 건수와 Lethal 비율을 확인한 결과 일수가 적은 2월은 사고 건수가 가장 적었습니다.

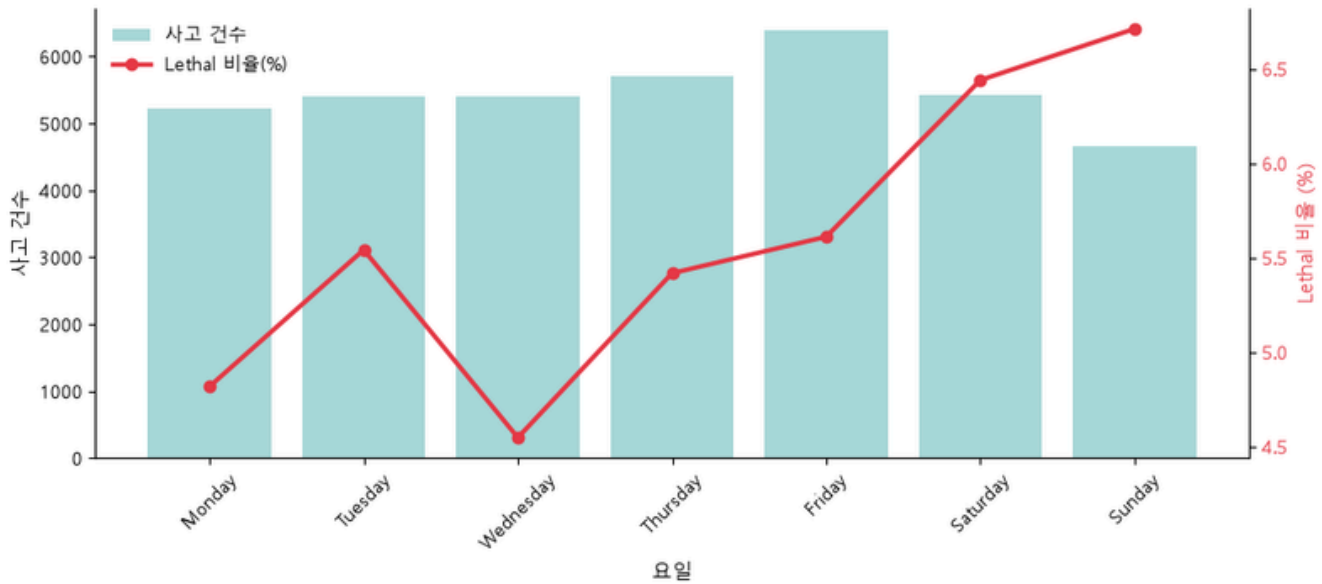
그러나 2월의 경우 사고 건수 대비 Lethal 비율은 오히려 가장 높게 나타났습니다.

3월부터 6월까지의 전체 사고 건수가 뚜렷하게 증가하는 반면 Lethal 비율은 하락하는 추세를 보입니다.

다만 이 구간의 Lethal 절대 건수를 확인한 결과 158건에서 183건으로 오히려 증가했으며
전체 사고 건수의 증가폭이 더 컸기 때문에 비율이 상대적으로 낮아진 것으로 나타났습니다.

즉, 이 구간은 실제 위험도가 낮아졌다기보다 사고 총량 증가에 비율이 희석된 결과로 해석하였습니다.

요일별 사고 건수 및 Lethal 비율



[19]: # 절대 건수(Lethal count) 요일별 확인
 daily_lethal_count = accidents_train.groupby('DayOfWeek')['Gravity'].apply(lambda x: (x == 1).sum()).reindex(day_order)
 print(daily_lethal_count)

```
DayOfWeek
Monday      252
Tuesday     300
Wednesday   246
Thursday    309
Friday      359
Saturday    350
Sunday      313
Name: Gravity, dtype: int64
```

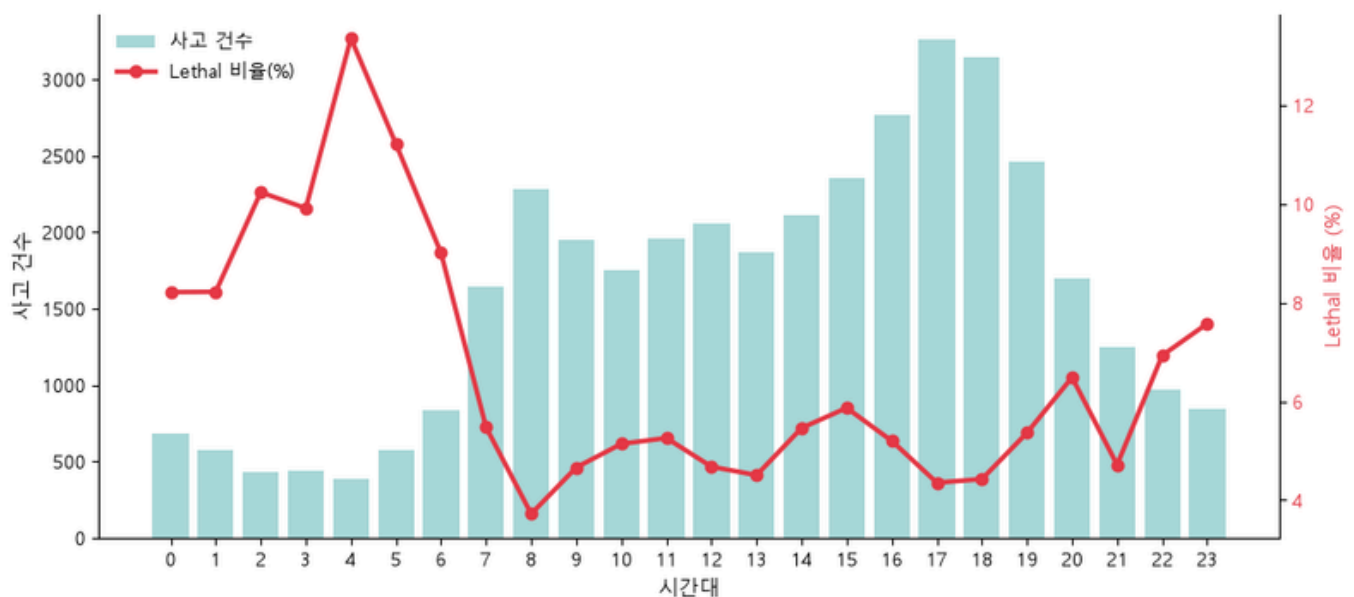
요일별 사고 건수를 보면 대체로 비슷한 수준이지만 수요일은 전체 건수 대비 Lethal 건수 자체가 가장 낮아 비율에서도 낮은 것을 확인할 수 있으며 심각도가 뚜렷하게 낮게 나타납니다.

금요일은 사고 건수와 Lethal 비율이 모두 평일보다 높게 나타나 뚜렷한 위험 신호를 보입니다.

반면 토요일과 일요일은 사고 건수 자체는 평일보다 낮아지지만 Lethal 비율은 오히려 평일보다 높게 나타납니다.

이는 주말에는 사고 발생 빈도와 무관하게 한 번 사고가 발생했을 때 치명적일 가능성이 더 높아진다는 것을 의미합니다.

시간대별 사고 건수 및 Lethal 비율



시간대별 그래프를 확인한 결과 통행량이 적은 22시~06시 구간은 사고 발생 건수 자체도 낮게 나타나지만 이 구간의 Lethal 비율은 통행량이 많은 8시~19시 구간보다 오히려 높게 나타납니다

```
[22]: # 절대 건수(Lethal count) 시간대별 확인
hourly_lethal_count = accidents_train.groupby('HourOfDay')['Gravity'].apply(lambda x: (x == 1).sum())
print(hourly_lethal_count)
```

```
HourOfDay
0      56
1      47
2      44
3      43
4      51
5      64
6      75
7      90
8      85
9      91
10     90
11    103
12     96
13     84
14    115
15    138
16    144
17    142
18    139
19    132
20    110
21     59
22     67
23     64
Name: Gravity, dtype: int64
```

```
*[24]: # 시간대별 전체 건수
hourly_total = accidents_train['HourOfDay'].value_counts().sort_index()

# 야간(22~06시) vs 주간 비율/건수 비교
night_hours = [22, 23, 0, 1, 2, 3, 4, 5, 6] # 22시~06시
day_hours = list(range(8, 20)) # 8시~19시

night_ratio_range = (hourly_lethal_ratio[night_hours].min(), hourly_lethal_ratio[night_hours].max())
day_ratio_range = (hourly_lethal_ratio[day_hours].min(), hourly_lethal_ratio[day_hours].max())

night_count_range = (hourly_total[night_hours].min(), hourly_total[night_hours].max())
day_count_range = (hourly_total[day_hours].min(), hourly_total[day_hours].max())

print("야간 비율 범위:", night_ratio_range, "/ 주간 비율 범위:", day_ratio_range)
print("야간 건수 범위:", night_count_range, "/ 주간 건수 범위:", day_count_range)

야간 비율 범위: (np.float64(6.928645294725956), np.float64(13.350785340314136)) / 주간 비율 범위: (np.float64(3.7231712658782308), np.float64(5.86734693877551))
야간 건수 범위: (np.int64(382), np.int64(967)) / 주간 건수 범위: (np.int64(1751), np.int64(3261))
```

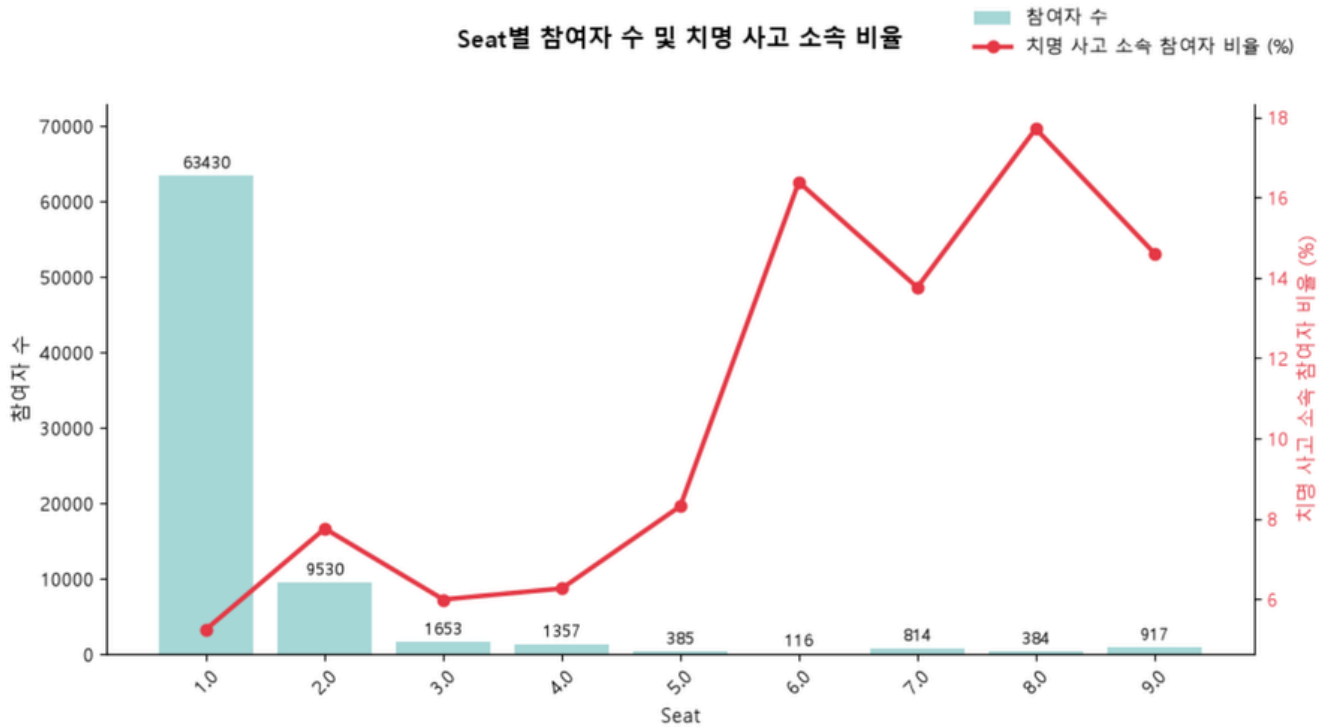
야간과 주간의 사고 특성을 비교하기 위해 시간대별 사고 건수와 Lethal 비율을 함께 확인했습니다.
 22시~06시의 시간대별 Lethal 비율은 약 6.9~13.4%이며 08시~19시는 약 3.7~5.9%로 나타났습니다.
야간에는 관측된 사고 건수가 적었지만 발생한 사고 중 치명 사고가 차지하는 비율은 주간보다 높다는 것을 보여줍니다.
 즉, 관측된 데이터에서는 야간 사고의 치명 비율이 더 높았다는 것을 알 수 있습니다.

Date · Hour 결론

기존 컬럼은 삭제처리하고 새로 생성한 월, 요일, 시간 컬럼을 사용합니다.
 EDA에서 확인한 핵심 패턴인 심야시간과 주말의 특성을 모델 입력에 반영하기 위해 이진 플래그로 추가하겠습니다.
 RandomForest와 XGBoost 같은 트리 기반 모델은 시간대별 비선형 패턴을 스스로 분기하여 학습할 수 있기 때문에 새로 생성한 시간 컬럼은 그대로 유지합니다.

3. RoadUserType 컬럼 생성

RoadUserType 컬럼은 사고에 관련된 인물이 운전자, 탑승객, 보행자 중 어디에 해당하는지 구분하는 컬럼입니다.
이 변수는 Seat 컬럼과 보행자 관련 컬럼을 활용해 파생한 변수입니다.
탑승객 인원과 보행자 포함 여부는 Lethal 여부에 영향을 줄 수 있는 요인이므로 추후 사고 단위로 데이터를 결합할 때 Seat 정보와 보행자 관련 컬럼의 정보가 희석되지 않도록 이 변수를 미리 생성했습니다.



Seat별 참여자 수를 확인한 결과 Seat 1과 2에 대부분이 집중되어 있으며 Seat 3번 이후로는 건수가 크게 줄어듭니다.
Lethal 비율은 Seat 1 ~ 5까지는 약 5%에서 8% 수준으로 비교적 낮게 유지되지만
Seat 6번 이후부터는 약 14%에서 18% 수준으로 뚜렷하게 높아지는 경향을 보입니다.
Seat 5번 이후 구간은 건수 자체가 매우 적어 비율이 우연히 높게 나타났을 가능성도 있으므로
이후 탑승 인원수나 보행자 포함 여부 등 다른 변수와 함께 교차 확인을 진행하기 위해 'RoadUserType'를 생성합니다.

RoadUserType 컬럼 생성 로직

```
[86]: # 보행자 판별 플래그 생성
users_train['is_pedestrian'] = (
    users_train['PedestrianLocation'].notna() |
    users_train['PedestrianAction'].notna() |
    users_train['PedestrianCompany'].isin(['Alone', 'WithSomebody', 'WithGroup']))
```

Seat 값이 결측인 사람들의 역할(운전자/동승자/보행자)을 추론해서 채우는 단계입니다.
우선 PedestrianLocation·PedestrianAction 중 하나라도 값이 있거나 PedestrianCompany가 'Unknown'이 아닌 의미 있는 값인 경우 보행자로 판별합니다.

```
[87]: # 그룹 키 정의 : Seat 결속치를 채울 때 같은 차 안에서 누가 운전자인가를 판단하는 단위가 된다.
group_key = ['AccidentId', 'VehicleId']

# 이미 운전자가 확정된 그룹 집합 생성
driver_groups = set(users_train.loc[users_train['Seat'] == 1, group_key].apply(tuple, axis=1))

# 역할 추론이 필요한 대상 지정 : Seat 값이 없고 보행자 관련 컬럼에도 값이 없는경
target_mask = users_train['Seat'].isna() & (~users_train['is_pedestrian'])

# 같은 그룹 안에서 Seat 결속 및 비보행자인 사람이 몇 명인지 수를 구한다.
# 이 인원이 1명인 경우 운전자로 추정할 근거가 된다.
group_size = users_train.loc[target_mask].groupby(group_key)[group_key[0]].transform('size')
users_train.loc[target_mask, 'target_group_size'] = group_size
```

다음은 그룹 키를 정의하는 단계입니다. 하나의 사고에는 차량이 여러 대 연관될 수 있기 때문에 '사고 단위'가 아닌 '사고 내 차량 단위'로 인원을 판단할 수 있는 기준을 생성하며 이렇게 만든 그룹 키는 이후 두 가지 판단에 사용됩니다. 첫째, 같은 그룹 안에 이미 운전자가 확정된 사람이 있다면 그룹에서 Seat 값이 없는 나머지 인원은 동승자로 판단합니다. 둘째, 같은 그룹 안에서 보행자 정보도 없고 확인된 운전자가 없이 결속 인원이 1명인 경우 운전자로 판단합니다.

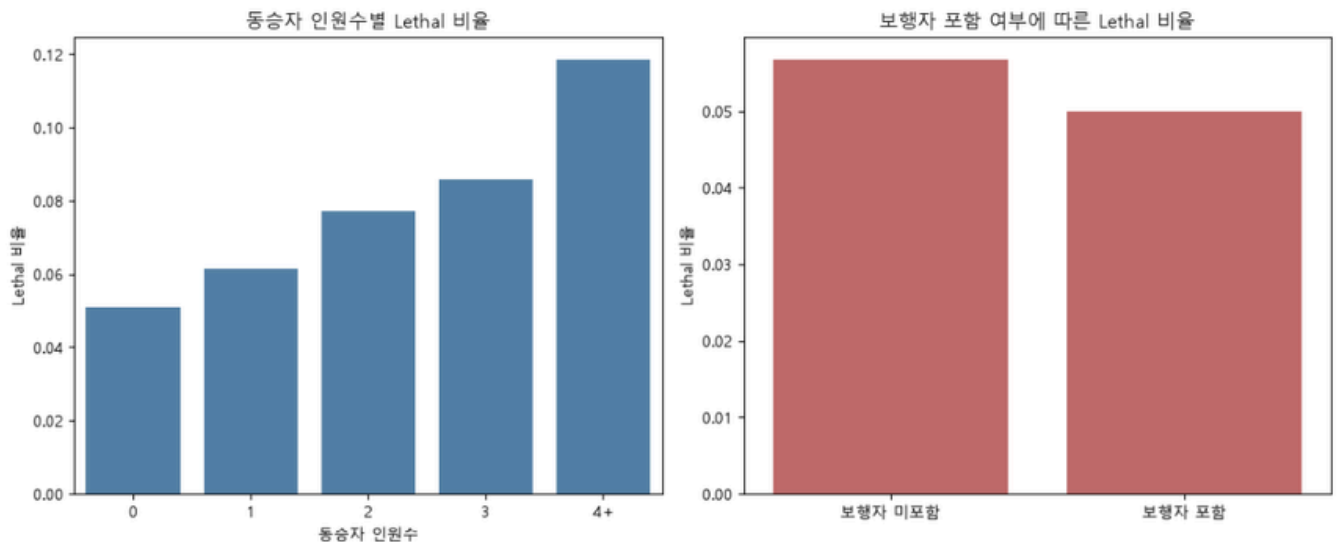
```
[88]: # 최종 RoadUserType 생성하여 라벨링 함수 생성하기
def get_role(row):
    if pd.notna(row['Seat']):
        return 'Driver' if float(row['Seat']) == 1 else 'Passenger'
    elif row['is_pedestrian']:
        return 'Pedestrian'
    key = (row['AccidentId'], row['VehicleId'])
    if key in driver_groups:
        return 'Passenger' # 이미 운전자가 있으면 무조건 동승자로 (변경하지 않음)
    elif row.get('target_group_size') == 1:
        return 'Driver'
    else:
        return 'Unknown'
```

```
[89]: #최종 RoadUserType 생성 및 라벨 저장
users_train['RoadUserType'] = users_train.apply(get_role, axis=1)

# 분류 결과 확인
print(users_train['RoadUserType'].value_counts(dropna=False))

RoadUserType
Driver      64652
Passenger   15350
Pedestrian   6986
Unknown      4
Name: count, dtype: int64
```

최종적으로 get_role 함수를 생성하여 운전자·동승자·보행자를 구분하는 RoadUserType를 생성하였습니다. 다음으로는 동승자 인원수에 따른 Lethal의 비율과 보행자를 포함한 사고에서의 Lethal 비율을 확인하겠습니다.



```
동승자 인원수별 Lethal 비율
PassengerCountGroup
0      0.050850
1      0.061341
2      0.077181
3      0.085919
4+     0.118699
Name: Gravity, dtype: float64
```

```
보행자 포함 여부에 따른 Lethal 비율
HasPedestrian
False    0.056804
True     0.049901
Name: Gravity, dtype: float64
```

동승자 인원수에 따른 Lethal 비율은 인원이 늘어날수록 함께 상승하는 것으로 확인됩니다. 다만 동승자 수 자체가 치명사고 비율을 높인다기보다는 동승자가 많을수록 탑승객이 많은 대형 차량 사고일 확률이 높고 그만큼 위험도가 상승했을 가능성으로 해석해볼 수 있습니다. 이를 명확히 하기 위해서는 차종 등 다른 컬럼과의 관계를 함께 살펴볼 필요가 있습니다.

보행자 포함 여부에 따른 Lethal 비율을 보면 보행자가 미포함된 사고에서 더 높은 치명률이 관측되었습니다. 이는 보행자가 다니는 도로에서는 상대적으로 치명사고로 이어질 구조적 요인이 적을 수 있다는것을 시사할 수 있으며 도로 유형 · 속도 환경 · 차량 구성 등의 차이에 따른 결과인지 추가적인 확인이 필요합니다.

Seat 컬럼 결론

Seat 컬럼 대신 새로 생성한 RoadUserType 컬럼을 모델의 피처 구성에 활용하겠습니다.

이는 Users 테이블을 AccidentId 기준으로 집계·결합하는 과정에서 운전자·동승자·보행자 등 도로 이용자 유형별 정보를 보존하고 이를 Lethal 분류에 활용하기 위함입니다. 현재 단계에서는 RoadUserType을 범주형 컬럼으로 유지하고 결합 데이터 생성 단계에서 원-핫 인코딩한 뒤 사고별로 합산하여 각 유형의 인원수를 나타내는 피처를 생성하겠습니다.

이외 에도 EDA 단계에서는 각 컬럼의 분포와 특성값별 Lethal 비율을 살펴보고 그 결과를 바탕으로 범주를 재구성 및 파생변수를 생성하는 작업을 하였습니다. 이 과정에서는 데이터의 특성을 파악하는 것뿐만 아니라 추후 AccidentId 기준으로 집계·결합할 때에도 해당 정보가 유지되도록 피처를 설계하는 데 중점을 두었습니다. 이를 위해 도로 이용자 유형별 인원수와 차량 수 등을 사고 단위의 피처로 구성하고자 하였습니다.

05. Accident-Level Aggregation

사고 환경과 참여자 및 차량의 특성을 통합하여
사고당 한 행의 모델 입력 데이터를 구성하였습니다.

Users 사고 참여자 구성 집계

집계 대상	사고별 집계 기준
전체 인원	해당 사고의 Users 행 수
운전자·동승자·보행자	RoadUserType별 인원수
연령·성별·이동 목적	각 범주에 해당하는 인원수
안전장치 종류·사용 상태	각 범주에 해당하는 인원수
연령 결측	연령을 알 수 없는 인원수
차량 수	중복을 제외한 VehicleId 수
차량 유형	Category별 포함 여부를 0과 1로 표현

사람 단위 기록을 AccidentId 기준으로 집계하여
사고당 한 행으로 구성하였습니다.

정보의 의미에 따라 인원수·고유 차량 수·차량 유형
포함 여부를 구분하여 피처를 생성하였습니다.

Vehicles 사고 관련 차량 특성 집계

집계 대상	사고별 집계 기준
진행 방향 (Direction)	진행 방향별 해당 차량 수
고정 장애물 (FixedObstacle)	고정 장애물 유형별 해당 차량 수
이동 장애물 (MobileObstacle)	이동 장애물 유형별 해당 차량 수
충돌 부위 (ImpactPoint)	충돌 부위별 해당 차량 수
기동 유형 (Maneuver_grouped)	기동 유형별 해당 차량 수

차량별 범주형 특성을 원-핫 인코딩한 뒤
AccidentId 기준으로 합산하였습니다.

이를 통해 각 특성에 해당하는 차량 수를 산출하고
여러 차량의 특성을 사고 단위로 표현하였습니다.

최종 사고 단위 데이터 결합

```
[29]: train_final.head(3)
```

```
[29]:
```

	AccidentId	Gravity	Month	HourOfDay	IsWeekend	IsNight	PeopleCount	DriverCount	PassengerCount	PedestrianCount	...	Infrastruct
0	201800023440	0	2	7	0	0	2	1	0	1	...	
1	201800041166	0	10	8	0	0	2	1	0	1	...	
2	201800040998	0	12	3	0	1	2	2	0	0	...	

3 rows × 194 columns

최종 결합 단계에서는 Accidents를 기준으로 Places의 도로·장소 정보와 사고 단위로 집계한
Users의 참여자 구성 Vehicles의 차량 특성을 AccidentId로 결합하였습니다.

이를 통해 사고 환경과 참여자·차량 정보가 사고당 한 행에 담긴 모델 입력 데이터를 구성하였으며
Train과 Test에 동일한 결합 방식을 적용하였습니다.

결합 후 Accidents와 Places의 범주형 변수에 원-핫 인코딩을 적용하여 모델에 입력할 수 있는 수치형 피처로 변환하였습니다.

최종 입력 피처 192개 · Train 38,256건 · 내부 평가용 Test 9,565건

06. Model Training & Evaluation

최종적으로 선택한 모델의 선정이유와
모델의 성능 및 결론에 대해서 정리하였습니다.

1. 모델 선정 이유

최종 데이터는 도로 유형, 기상 상태, 충돌 유형 등 범주형 변수와 사고 단위 집계 변수를 중심으로 구성되어 있습니다.

로지스틱 회귀는 계수를 통해 변수별 관계를 해석할 수 있다는 장점이 있지만 기본 형태에서는 각 변수의 효과가 로그오즈에 가산적으로 반영되기 때문에 도로 환경과 사고 참여자 특성 등 여러 조건이 결합하는 효과를 표현하려면 상호작용항을 별도로 설계해야 한다는 한계가 있습니다.

이에 따라 변수별 조건을 분기하면서 비선형 관계와 변수 간 상호작용을 학습할 수 있는 트리 기반 모델을 선택하였습니다. 또한 다른 앙상블 학습 방식의 성능을 비교하기 위해 RandomForest와 XGBoost를 사용하였습니다.

- **RandomForest**: 복원추출한 표본과 무작위 후보 피쳐로 여러 트리를 학습해 예측을 종합하는 배깅 기반 모델
- **XGBoost**: 이전 트리의 손실을 줄이는 방향으로 트리를 순차적으로 추가하는 부스팅 기반 모델

최종적으로 두 모델을 비교해 현재 데이터에서 어떤 학습 방식이 치명 사고 분류에 더 적합한지 확인해보았습니다.

2. 기준모델 설정 및 평가방법

기준모델과 교차검증

전체 사고 중 Lethal의 비율은 약 5.6%로 모든 사고를 NonLethal로 예측한다고 해도 높은 정확도를 얻을 수 있습니다. 불균형한 분포에서 정확도만으로 모델을 평가하는 것이 적절한지 확인하기 위해 다수 클래스인 NonLethal을 예측하는 DummyClassifier를 기준모델로 설정하였습니다.

기준모델을 학습한 결과 Accuracy는 94.44%였으며 Lethal Recall과 F1은 모두 0으로 확인되었습니다. 이를 통해 높은 정확도가 치명 사고를 잘 탐지한다는 의미는 아니라는 것을 알 수 있었으며 이후 모델들은 Precision, Recall, F1, F2, ROC-AUC, Average Precision을 함께 평가하였습니다.

또한 Lethal의 표본이 적어 한 번의 데이터 분할에 평가가 의존하지 않도록 Stratified 5-Fold 교차검증을 적용하여 전체 데이터의 클래스 비율을 그대로 반영하였으며 두 모델을 비교할 때도 동일한 5개의 Fold를 사용하였습니다. 이를 통해 생성된 5번의 학습 · 검증 조합에서 평균 성능을 산출하여 비교하였습니다.

```
dummy = DummyClassifier(strategy='prior')
dummy_cv = cross_validate(dummy, X_train, y_train, cv=5, scoring=baseline_scoring)
print('[DummyClassifier 기준선]')
for metric in baseline_scoring:
    print(f"{metric}: {dummy_cv[f'test_{metric}'].mean():.4f}")
```

```
[DummyClassifier 기준선]
accuracy: 0.9444
recall: 0.0000
f1: 0.0000
f2: 0.0000
roc_auc: 0.5000
AP: 0.0556
```

3. RandomForest 학습 및 임계값 조정

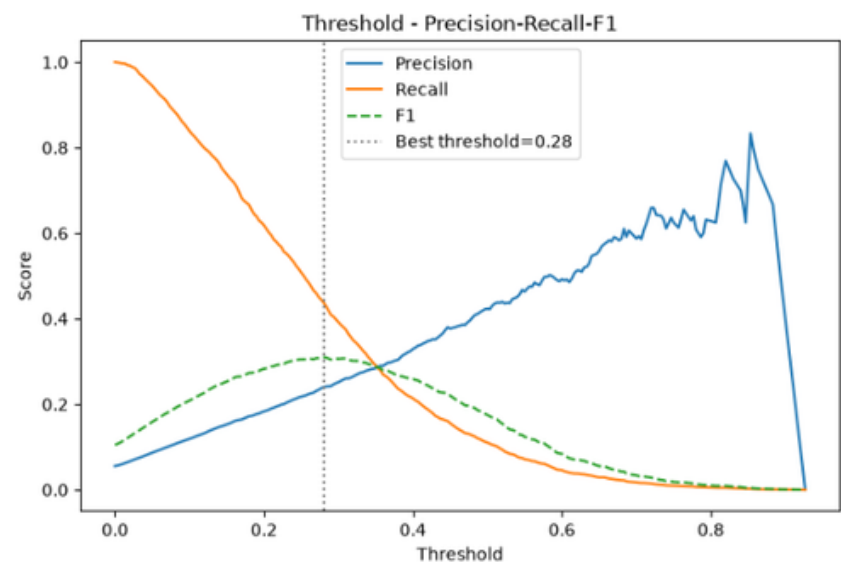
```
# 평균 점수 확인
for metric in scoring :
    print(f'{metric}: {rf_cv[f"test_{metric}"].mean():.4f}') # 소수점 4자리까지만 표기

accuracy: 0.9422
precision: 0.4237
recall: 0.1076
f1: 0.1716
f2: 0.1265
roc_auc: 0.8228
AP: 0.2453
```

클래스 불균형을 고려하여 class_weight='balanced'를 적용한 RandomForest로 교차검증을 진행하였습니다. RandomForest로 교차검증 결과 기본 임계값 0.5에서 Recall은 0.1076 F1은 0.1716으로 클래스 가중치를 적용했지만 실제 Lethal 사고를 충분히 탐지하지 못하는 문제가 확인되었습니다.

다음으로 Lethal로 분류하는 임계값의 기준을 조정하면 탐지 성능이 개선될 수 있는지 확인해보았습니다.

OOF 예측을 활용한 임계값 선정



각 표본을 학습에 사용하지 않은 Fold의 모델로 예측을 진행하여 OOF 예측 확률을 생성하였습니다. 이를 바탕으로 임계값별 Precision, Recall, F1을 비교하고 F1이 최대가 되는 임계값 0.28을 선정하였습니다.

치명사고의 미탐을 중요하게 보지만 오경보도 함께 증가하기 때문에 Precision과 Recall의 균형을 나타내는 F1을 탐색 기준으로 사용하였으며 Recall을 더 중요하게 반영하는 F2는 보조 평가 지표로 확인하였습니다.

Test 데이터로 최종 성능 평가

	model	threshold	precision	recall	f1	f2	roc_auc	AP
0	RandomForest 기본	0.28	0.235	0.4192	0.3011	0.3624	0.8224	0.2326

앞서 Train을 활용해 교차검증과 임계값을 선정하였고 최종적으로 Test 데이터 9,565건으로 예측 확률을 계산했습니다.

RandomForest 모델이 산출한 예측 확률에 임계값 0.28을 적용하여 Lethal 여부를 분류하였으며 Test 데이터의 실제 정답과 비교한 결과 Recall 0.4192 F1은 0.3011을 기록하였습니다.

이 결과는 실제 Lethal 사고의 절반 이상을 놓치고 있다는 문제점이 있었으며 다른 모델을 사용하여 성능 개선이 가능한지 확인하기 위해 동일한 Train 데이터와 Test 데이터를 사용하여 XGBoost 모델을 학습 및 평가를 진행했습니다.

4. XGBoost 학습 및 임계값 조정

	model	threshold	precision	recall	f1	f2	roc_auc	AP
0	XGBoost 기본	0.3686	0.1989	0.4154	0.269	0.3412	0.775	0.1938

XGBoost에는 학습 데이터의 클래스 비율을 반영한 `scale_pos_weight=16.98`을 적용하였습니다. RandomForest와 동일한 Fold로 교차검증을 진행하였고 기본 임계값에서 Recall 0.2678 F1 0.2502을 기록하였습니다.

이는 기본 임계값을 사용한 RandomForest보다 높은 결과이지만 Lethal 탐지율은 여전히 개선이 필요했습니다. 따라서 XGBoost도 OOF 예측 확률을 이용하여 임계값 0.3686를 선정하였고 이후 전체 학습 데이터로 모델을 학습하고 별도 평가 데이터에 적용한 결과 Recall 0.4154, F1 0.269를 기록했습니다.

기본 임계값을 적용한 교차검증에서는 XGBoost의 Recall이 RandomForest보다 높았습니다. 그러나 각 모델의 OOF 예측으로 선정한 임계값을 적용한 Test 평가에서는 RandomForest의 Recall이 0.4192로 XGBoost의 0.4154보다 소폭 높았으며, F1도 0.3011로 XGBoost의 0.2690보다 높았습니다.

RandomForest를 최종 후보로 선정하고 추가적인 성능 개선 가능성을 확인하기 위해 RandomizedSearchCV를 이용한 하이퍼파라미터 튜닝을 진행하였습니다.

5. RandomForest 하이퍼파라미터 튜닝 및 최종 선정

	model	threshold	precision	recall	f1	f2	roc_auc	AP
0	RandomForest 기본	0.2800	0.2350	0.4192	0.3011	0.3624	0.8224	0.2326
2	RandomForest 튜닝	0.5826	0.2380	0.4004	0.2985	0.3523	0.8202	0.2426
1	XGBoost 기본	0.3686	0.1989	0.4154	0.2690	0.3412	0.7750	0.1938

RandomForest의 추가 개선 가능성을 확인하기 위해 RandomizedSearchCV로 트리 수, 최대 깊이, 분기에 필요한 최소 표본 수, 리프의 최소 표본 수, 분기 시 고려할 피쳐 수를 탐색 하였습니다.

튜닝 모델의 OOF 예측을 통해 선정한 임계값은 0.5826이며 Test 평가 결과 튜닝 전보다 Precision은 소폭 상승했지만 Recall과 F1은 감소하였습니다. 이는 오경보가 줄어드는 대신 실제 치명 사고를 놓치는 경우가 증가했음을 의미합니다.

결과적으로 이번 하이퍼파라미터 탐색 범위에서는 Recall과 F1의 개선을 확인하지 못하였으며 클래스 가중치를 적용한 튜닝 전 RandomForest 모델과 임계값 0.28을 최종 모델 및 분류 기준으로 선택하였습니다.

RandomForest가 더 좋은 성능을 보인 이유에 대한 해석

RandomForest는 서로 다른 표본과 피쳐 선택을 활용해 여러 트리를 학습하고 예측을 평균화함으로써 개별 트리의 변동성을 줄이는 반면 XGBoost는 현재 모델의 손실을 줄이는 방향으로 트리를 순차적으로 추가합니다. RoadSense 데이터는 희귀 범주를 포함한 원-핫 인코딩 피쳐가 다수이고 Lethal 표본이 제한적이므로 이러한 조건에서 RandomForest의 예측 평균화가 일부 표본에 대한 민감도를 완화하는 데 도움이 되었을 가능성이 있다고 판단하였습니다.

6. 오버샘플링을 통한 성능 개선 가능성 검토

Lethal 표본 부족으로 인해 모델이 분류를 잘 못한다는 가설을 확인하기 위해 SMOTENC를 적용해보았습니다. SMOTENC는 범주형 변수와 수치형 변수를 함께 고려해 선형보간 방식으로 오버샘플링을 수행합니다. 최종 데이터셋에는 범주형 변수와 인원 수·차량 수 등의 집계 변수가 함께 존재하므로 이 방법이 적절하다고 판단했습니다. 이에 따라 성능뿐 아니라 합성 표본 자체의 타당성도 함께 점검했습니다.

비교 조건을 통일하기 위해 베이스라인과 동일한 RandomForest 설정과 5-Fold 분할을 사용했습니다. 각 Fold의 학습 부분에만 증강을 적용했고 임계값은 Train의 OOF 예측으로 선정한 뒤 Test 데이터에서 평가했습니다.

	model	threshold_policy	threshold	accuracy	precision	recall	f1	f2	roc_auc	ap	tn	fp	fn	tp
1	A_RF	OOF_F1	0.1533	0.8941	0.2289	0.3816	0.2861	0.3367	0.8124	0.2202	8349	684	329	203
3	B_RF_weight	OOF_F1	0.1367	0.8859	0.2262	0.4342	0.2975	0.3668	0.8207	0.2327	8243	790	301	231
5	C_SMOTENC	OOF_F1	0.2400	0.8980	0.2363	0.3741	0.2897	0.3350	0.8136	0.2116	8390	643	333	199
7	D_SMOTENC_weight	OOF_F1	0.2067	0.8765	0.1924	0.3816	0.2558	0.3189	0.8043	0.1854	8181	852	329	203
9	E_ROS	OOF_F1	0.2233	0.9028	0.2519	0.3797	0.3028	0.3447	0.8221	0.2327	8433	600	330	202

가중치가 없는 조건에서는 SMOTENC 적용 전후 성능이 비슷해 뚜렷한 개선을 확인하지 못했습니다. 가중치를 함께 적용한 조건에서는 오히려 F1이 감소했으며 단순 복제는 수치상 가장 높았지만 SMOTENC보다 확실히 우수하다고 단정하기는 어려웠습니다.

또한 SMOTENC로 생성된 행의 약 99.8%에서 카운트 값이 정수가 아닌 소수 형태로 나타났고 약 1.0%에서는 보행자 역할 인원과 플래그가 서로 일치하지 않는 문제가 발견되었습니다. 이는 수치 보간과 범주 생성 과정이 사고 집계 데이터의 의미를 온전히 유지하지 못할 수 있음을 보여줍니다.

따라서 이번 실험에서는 오버샘플링의 안정적인 개선 효과를 확보하지 못했으며 집계 데이터에서는 표본 수뿐 아니라 합성값과 변수 간 관계의 타당성도 함께 고려해야 한다고 판단했습니다.

7. 종합 결론 및 활용 방향

종합 결론

현재 데이터와 실험 범위에서는 개별 사고의 치명 여부를 안정적인 모델의 예측으로 판정하기 어렵다고 판단하였습니다. 다만 변수별 치명률 분석에서 확인한 사고 조건은 안전관리의 우선 조사 및 점검 대상을 선정하는 참고자료로 사용할 수 있으며 이를 바탕으로 개선 가능한 요인을 검토하는 데 활용할 수 있습니다. 또한 사고 발생시 피해 심각도를 낮추기 위한 의사결정을 지원하는 데도 도움이 될 것으로 보았습니다.

한계점 및 보완 방향

- 높은 치명률이 관측되었다 해서 해당 조건이 직접적인 요인이라는 것은 아닙니다. 범주별 표본 수와 다른 조건의 영향을 함께 검토하여 요인을 개선한 조치의 효과는 별도로 검증이 필요합니다.
- 충돌 유형 등은 사고 이후에 확인되는 변수로 사고전 위험 예측에 활용하기 위해서는 사고 이전에 확보 가능한 요인들을 변수로 사용하여 모델을 다시 구성하고 평가해야 합니다.